

1) Read a dataset from the user and i. Use the Find-S algorithm to find the most specific hypothesis that is consistent with the positive examples. Ii. What is the final hypothesis after processing all the positive examples? Using the same dataset, apply the Candidate Elimination algorithm. Determine the final version space after processing all examples (both positive and negative). What are the most specific and most general hypotheses in the version space?

Program 1: Find-S Algorithm

```
import pandas as pd

# Read the dataset
data = pd.read_csv("EnjoySport.csv")

# Get attributes and target
concepts = data.iloc[:, :-1].values
target = data.iloc[:, -1].values

# Initialize hypothesis
hypothesis = None

# Apply Find-S Algorithm
for i in range(len(concepts)):
    if target[i] == "Yes":

        if hypothesis is None:
            hypothesis = list(concepts[i])

        else:
            for j in range(len(hypothesis)):
                if hypothesis[j] != concepts[i][j]:
                    hypothesis[j] = "?"
```

```
# Display the final hypothesis
print("Final Hypothesis:")
print(hypothesis)
```

Output:

Final Hypothesis:
['Sunny', 'Warm', '?', 'Strong', '?', '?']

Program 2: Candidate Elimination Algorithm

```
import pandas as pd
# Read dataset
data = pd.read_csv("EnjoySport.csv")

concepts = data.iloc[:, :-1].values
target = data.iloc[:, -1].values

# Initialize S and G
S = list(concepts[0])
G = [['?' for i in range(len(S))] for j in range(len(S))]

for i in range(len(concepts)):

    if target[i] == "Yes":

        # Generalize S
        for j in range(len(S)):
            if S[j] != concepts[i][j]:
                S[j] = '?'
                G[j][j] = '?'

    else:

        # Specialize G
```

```

    for j in range(len(S)):
        if S[j] != concepts[i][j]:
            G[j][j] = S[j]
        else:
            G[j][j] = '?'

# Remove empty hypotheses
G_final = []
for g in G:
    if g != ['?'] * len(S):
        G_final.append(g)

print("Specific Hypothesis (S):")
print(S)

print("\nGeneral Hypotheses (G):")
for g in G_final:
    print(g)

```

Output:

Specific Hypothesis (S):

['Sunny', 'Warm', '?', 'Strong', '?', '?']

General Hypotheses (G):

['Sunny', '?', '?', '?', '?', '?']

['?', 'Warm', '?', '?', '?', '?']

Dataset:

 jupyter enjoysport.csv Last Checkpoint: 11 months ago								
File Edit View Settings Help								
Delimiter: ,								
	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport	
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes	
2	Sunny	Warm	High	Strong	Warm	Same	Yes	
3	Rainy	Cold	High	Strong	Warm	Change	No	
4	Sunny	Warm	High	Strong	Cool	Change	Yes	

2) Read a supervised dataset and use bagging and boosting technique to classify the dataset. Indicate the performance of the model.

Program:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import BaggingClassifier, AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report

# Load dataset
iris = load_iris()
X, y = iris.data, iris.target

# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

# ---- Bagging ----
bagging = BaggingClassifier(
    estimator=DecisionTreeClassifier(),
    n_estimators=50,
    random_state=42
)
bagging.fit(X_train, y_train)
y_pred_bagging = bagging.predict(X_test)
bagging_acc = accuracy_score(y_test, y_pred_bagging)

# ---- Boosting (AdaBoost with SAMME to avoid warning) ----
boosting = AdaBoostClassifier(
    estimator=DecisionTreeClassifier(max_depth=1),
    n_estimators=50,
```

```

    algorithm="SAMME", # ✓ avoids FutureWarning
    random_state=42
)
boosting.fit(X_train, y_train)
y_pred_boosting = boosting.predict(X_test)
boosting_acc = accuracy_score(y_test, y_pred_boosting)

# ---- Results ----
print("Bagging Accuracy:", bagging_acc)
print("Boosting Accuracy:", boosting_acc)

print("\n--- Bagging Classification Report ---")
print(classification_report(y_test, y_pred_bagging,
target_names=iris.target_names))

print("\n--- Boosting Classification Report ---")
print(classification_report(y_test, y_pred_boosting,
target_names=iris.target_names))

```

Output:

Bagging Accuracy: 0.9333333333333333				
Boosting Accuracy: 0.9333333333333333				
--- Bagging Classification Report ---				
	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	15
versicolor	0.88	0.93	0.90	15
virginica	0.93	0.87	0.90	15
accuracy			0.93	45
macro avg	0.93	0.93	0.93	45
weighted avg	0.93	0.93	0.93	45
--- Boosting Classification Report ---				
	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	15
versicolor	0.88	0.93	0.90	15
virginica	0.93	0.87	0.90	15
accuracy			0.93	45
macro avg	0.93	0.93	0.93	45
weighted avg	0.93	0.93	0.93	45